

| Phase 2 Explanation

| What Was Learned: Undefined Behavior and AddressSanitizer

Phase 2 demonstrated a real strcpy buffer overflow and its detection.

A char buffer[8] can hold at most 7 characters plus the terminating null. The program calls strcpy(buffer, ABCDEFGHIJKLMNO) which is 15 characters plus null equals 16 bytes written into an 8 byte buffer. Because strcpy receives no destination size, it copies until null regardless of capacity.

Writing past buffer[8] is undefined behavior in C++. The standard imposes no requirements after the overflow.

On this system three valid outcomes were observed for the same out of bounds write:

With default fstack protector: stack smashing detected terminated EXIT 134

With fsanitize=address,undefined: ERROR AddressSanitizer stack buffer overflow WRITE of size 16 [32,40) buffer line 44 at offset 40 EXIT 1 ABORTING

With fno stack protector: EXIT 0 and Buffer contains ABCDEFGHIJKLMNO, silent corruption that appears to succeed

AddressSanitizer makes the undefined behavior visible during testing. The stack buffer overflow report identifies the error type, the 16 byte write, the 8 byte buffer declared at line 44, the overflow at offset 40, and the call site at line 58 in vulnerableFunction.

Phase 1 had visualized the same 8 byte boundary safely with std::min capped inside a 20 byte array, no real overflow. Phase 2 removed the cap and let strcpy cross that boundary for real, proving that input length and buffer capacity must be checked explicitly on every copy.

Files:

intermediate-2-vulnerable-operation.mp4: A 6 to 8 minute animation demonstrating an unchecked strcpy operation and AddressSanitizer diagnostic.

src/phase2_vulnerable_strcpy.cpp: The intentionally vulnerable C++ source used in the demonstration.

docs/phase2_explanation.md: A short explanation of what was learned about undefined behavior and AddressSanitizer.